LAB-03 NOTES – UVM TESTS, PHASES AND TOPOLOGY

1. Goal / Objective
Understand how a UVM test controls the verification environment and how UVM phases are executed. Concepts learned: uvm_test, build_phase(), end_of_elaboration_phase(), run_test(), topology printing, derived tests and +UVM_TESTNAME.

2. What We Had Before Starting Lab-03
From Lab-02 we had Driver, Sequencer, Agent and Environment. Missing pieces were Test control, phase understanding and test selection.

3. Coding Flow Followed
1. Create base_test
2. Create Environment inside Test
3. Learn build_phase()
4. Learn end_of_elaboration_phase()
5. Print UVM Topology
6. Create top.sv
7. Call run_test()
8. Create Derived Tests
9. Run Specific Tests using +UVM_TESTNAME

4. Step-by-Step Development

Step 1 : Create Base Test
Why? Test controls the entire verification environment.
Syntax:
class base_test extends uvm_test;
`uvm_component_utils(base_test)
yapp_env env;
endclass
Learning: Test is the root of the verification environment.

Step 2 : Create Environment from Test
Syntax:
env = yapp_env::type_id::create("env", this);
Learning: build_phase() is used for component creation.

Step 3 : Learn build_phase()
Purpose:
- Component creation
- Configuration
- Factory overrides
Learning: build_phase() builds the hierarchy.

Step 4 : Learn end_of_elaboration_phase()
Purpose:
Inspect completed hierarchy after creation.
Learning: Used to verify hierarchy before simulation starts.

Step 5 : Print UVM Topology
Syntax:
uvm_top.print_topology();

Learning: Topology shows actual UVM hierarchy.

Step 6 : Create top.sv
Syntax:
module top;
initial begin
run_test();
end
endmodule
Learning: run_test() starts all UVM phases.

Step 7 : Understand run_test()
run_test() -> Creates Test -> Builds Hierarchy -> Runs UVM Phases.

Step 8 : Create Derived Tests
Example:
class test2 extends base_test;
`uvm_component_utils(test2)
endclass
Learning: Multiple tests can reuse the same environment.

Step 9 : Test Selection
Command:
+UVM_TESTNAME=test2
Learning: Same environment, different tests.

5. UVM Phase Flow Learned
build_phase -> connect_phase -> end_of_elaboration_phase -> start_of_simulation_phase -> run_phase

build_phase = Create Components
connect_phase = Connect Components
end_of_elaboration_phase = Verify Hierarchy
run_phase = Execute Simulation

6. Testbench Evolution
Before Lab-03:
Environment -> Agent

After Lab-03:
Top -> Test -> Environment -> Agent -> Driver & Sequencer

7. Issues Faced and Solutions
Issue 1: Wrong Test Running
Cause: Incorrect +UVM_TESTNAME configuration.
Fix: Verify Makefile and simulation command line.

Issue 2: Understanding run_test().
Learning: run_test() creates the test, builds hierarchy and executes phases.

Issue 3: Understanding Topology.
Learning: Topology confirms all components were built correctly.

Issue 4: build_phase() Purpose.
Learning: UVM expects hierarchy creation during build_phase().

8. Interview Questions
Why extend from uvm_test?
What is build_phase used for?
What is end_of_elaboration_phase?
What does run_test() do?
Why use +UVM_TESTNAME?

9. Final Learning Summary
Test = Controls Environment
build_phase = Creates Components
Topology = Shows Hierarchy
run_test = Starts UVM
+UVM_TESTNAME = Selects Test

10. One-Line Revision
Lab-03 introduced UVM tests, phases, topology printing and test selection mechanisms, making the environment controllable, configurable and reusable.

Memory Trick
Lab-01 = Build the Packet
Lab-02 = Build the Roads
Lab-03 = Build the Controller